



# OTDR Module

## Product Technical Manual V4.0

Table of Contents

|                                                                 |    |
|-----------------------------------------------------------------|----|
| <u>1.</u>                                                       | 1  |
| 2.                                                              | 1  |
| 3.                                                              | 1  |
| 4.                                                              | 2  |
| 5.                                                              | 2  |
| 5.1                                                             | 2  |
| 5.2                                                             | 2  |
| 5.3                                                             | 3  |
| 6.                                                              | 4  |
| 6.1                                                             | 4  |
| 6.2                                                             | 4  |
| 6.3                                                             | 7  |
| 6.4                                                             | 9  |
| 6.5                                                             | 11 |
| 7.                                                              | 12 |
| 7.1                                                             | 12 |
| 7.2                                                             | 13 |
| 7.3                                                             | 15 |
| 8.                                                              | 15 |
| 8.1                                                             | 15 |
| 8.1                                                             | 15 |
| Appendix 1: OTDR Module 4.0 Technical Development Documentation | 18 |

---

## 1. Preface

The contents of this document are subject to change without notice. The Company accepts no liability for any errors contained herein or for any incidental or consequential losses arising from the provision or use of this material, or from the practicality of the information contained herein.

## 2. Safety Instructions

The following safety precautions must be observed at all times when using the product. Failure to follow the safety operating procedures set out below or to comply with the specific warnings contained in this manual will constitute a breach of the safety standards governing the design, manufacture and use of the product; the Company accepts no liability for any consequences arising therefrom.

- Working Environment

Relative humidity 5% - 85% (non-condensing), operating temperature - 5 to +55 ° C, storage temperature - 40 to +85 ° C.

- Check the polarity and voltage range of the power supply before switching on

Before powering on the product, please ensure that the power supply has the correct polarity and voltage, and that appropriate safety precautions have been taken.

- Do not use the product in explosive atmospheres

Do not use this product in explosive environments, such as those containing flammable gases or vapours.

- Do not dismantle the product yourself

Do not attempt to dismantle the product casing or replace internal components yourself. If necessary, please contact our specialist maintenance staff.

## 3. Warnings



- The OTDR module is a laser device; users must always avoid looking directly into the laser output port. Do not use using microscopes, magnifying glasses or similar devices to observe the light source output port; if the laser beam's energy is focused onto the retina, causing permanent damage to the eyes.

- Before using the OTDR module to measure optical fibre, please check whether there is any active signal present in the fibre under

---

test. The presence of an active signal may lead to inaccurate measurement results and, in severe cases, cause permanent damage to the module (unless the module is equipped with an optical filter).

## 4. Precautions



**Power supply:** This OTDR module is powered by an external DC power supply, which must meet the following requirements:

Rated voltage 6V - 12V DC, with voltage ripple less than 20mV.

**Caution regarding laser radiation:** During measurements on fibre-optic systems, avoid looking directly at open-ended fibres,

fibre optic connectors, fibre optic splice points or other light sources; otherwise, the eyes may be exposed to transmitted laser and cause eye damage.

- When the OTDR module is in operation, do not look directly into the laser output port.

- After using the OTDR module, the dust cap on the fibre optic connector must be replaced.

- Do not look directly at the end of a fibre under test. If possible, point the end of the fibre towards a non-reflective object.

## 5. Overview

### 5.1 Contents of this Manual

Firstly, thank you for choosing our company' s product. Before use, please read the safety instructions, warnings and precautions in this manual carefully to avoid personal injury or damage to the equipment resulting from improper operation. This manual primarily covers the product' s features, interface definitions, standard operating procedures, technical specifications and maintenance methods.

### 5.2 Unpacking and Inspection

This product has been packed and shipped in accordance with standard assembly and freight procedures. Upon receipt of the goods, please carefully check the names and quantities of the product and accessories against the packing list, and inspect the external condition of the product. This will enable you to promptly identify any physical damage that may have occurred during transit. Should you discover any damage to the packaging, please contact the freight company and our authorised

---

distributor immediately to resolve the matter in accordance with the relevant procedures set out in the freight agreement. Should the actual product or accessories (such as the power adapter, PC analysis software installation CD and technical specifications) differ in name or quantity from those listed on the packing list, please contact the product's supply agent promptly to resolve the matter. If necessary, you may also contact our company's after-sales service technicians directly.

### **5.3 Product Introduction**

The full English name of OTDR is Optical Time Domain Reflectometer. The OTDR module is a precision opto-electronic integrated module manufactured utilising the backscatter generated by Rayleigh scattering and Fresnel reflection when light propagates through optical fibre. It is widely used in the maintenance and installation of optical cable lines and can measure fibre length, transmission attenuation, splice attenuation and fault location.

The OTDR module is part of a series of high-dynamic-range, high-performance OTDR modules. It primarily integrates core components such as OTDR pulse generation, reception and amplification, as well as data acquisition and analysis processing. It provides an Ethernet interface, facilitating easy secondary development. This product is compact, cost-effective, convenient and efficient, and can be used to identify actual faults in various fibre-optic links. It is widely applicable in the installation, daily maintenance and fault analysis and diagnosis of fibre-optic cables.

The module connects to a PC (or embedded system) via an Ethernet interface. By running PC-based application software on the PC, communication between the PC and the module is enabled. In accordance with the communication protocol, this facilitates functions such as the configuration of OTDR measurement parameters, the transmission of commands, the uploading of test curves, and the display of data analysis results.

Users can utilise the Ethernet interface communication protocol to carry out secondary development of the module, enabling them to develop their own OTDR instruments or fibre-optic online monitoring systems with the utmost speed. If users utilise a mature application platform for their instrumentation products, they need only develop the human-machine interface and embed the OTDR module into their instrumentation to create a new OTDR instrument, thereby rapidly capturing market

---

share; users may also embed the OTDR module into other optical communication equipment or system products to add OTDR testing and analysis capabilities to those products.

Product features:

- High dynamic range, small blind spot
- Plug-and-play via Ethernet interface
- Fast and efficient data transmission, analysis and storage
- Functionality for comparing test curves and historical curves
- Automatic OTDR curve analysis with clear and concise measurement information

Main areas of application:

- Fibre optic cable acceptance testing
- PON application testing
- Remote fibre monitoring
- Fibre-optic sensor integration components
- Integrated components for optical transmission equipment

## **6. Product Overview**

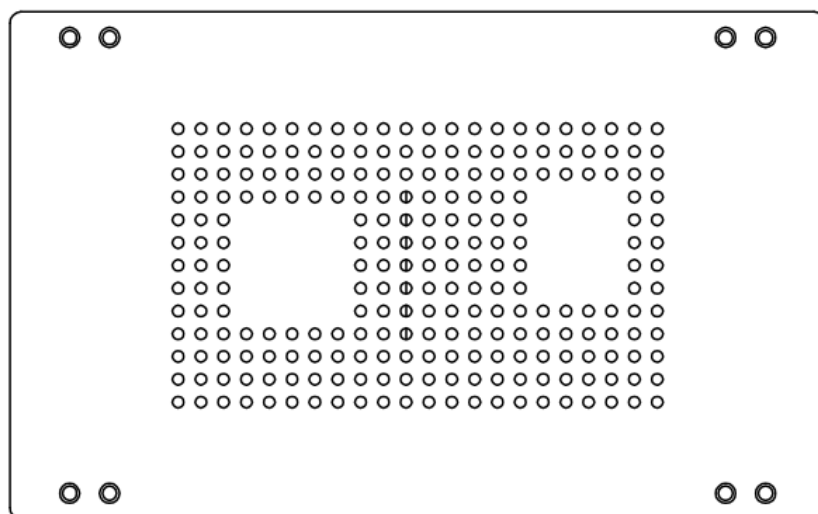
### **6.1 Introduction**

This section provides an overview of the OTDR module ( ), covering its physical structure, interface definitions, standard operating procedures and other relevant information, including precautions to be observed during use. Should you encounter any issues whilst using the product, please contact the technical support team at the company or one of its authorised distributors.

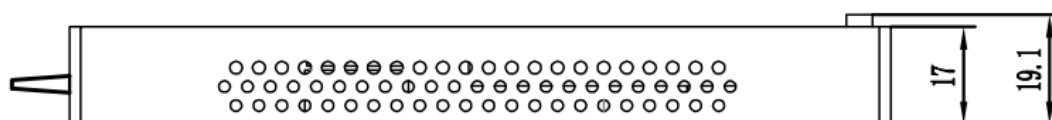
### **6.2 Product Structure and Appearance**

The OTDR module features a metal casing with an anodised finish. To ensure adequate heat dissipation for the internal components, ventilation holes have been incorporated into the sides of the casing. The specific dimensions of the casing design are shown in Figure 1.

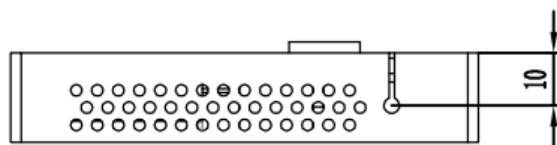
顶视图



侧视图



侧视图 (光接口)



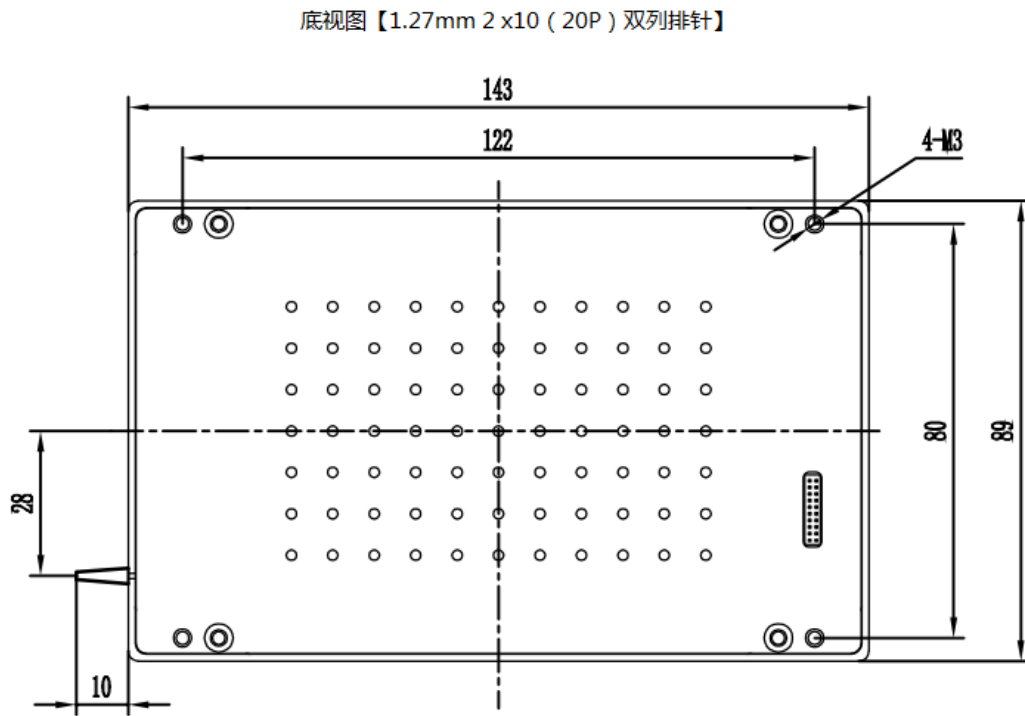
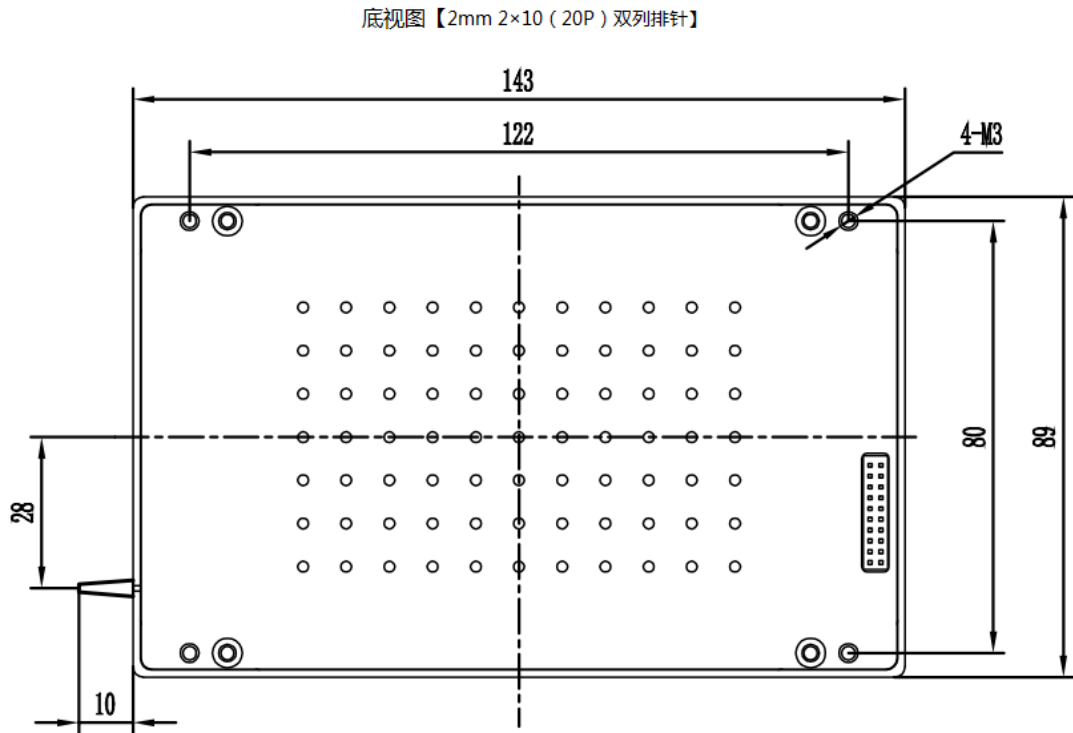


Figure 1: Housing Dimensions (in mm)

The OTDR module features a compact and space-saving design; it can be used independently or easily integrated into other equipment. The physical appearance of the module's electrical interface [2 mm 2×10 (20P) dual-row pin header] is shown in Figure 2.



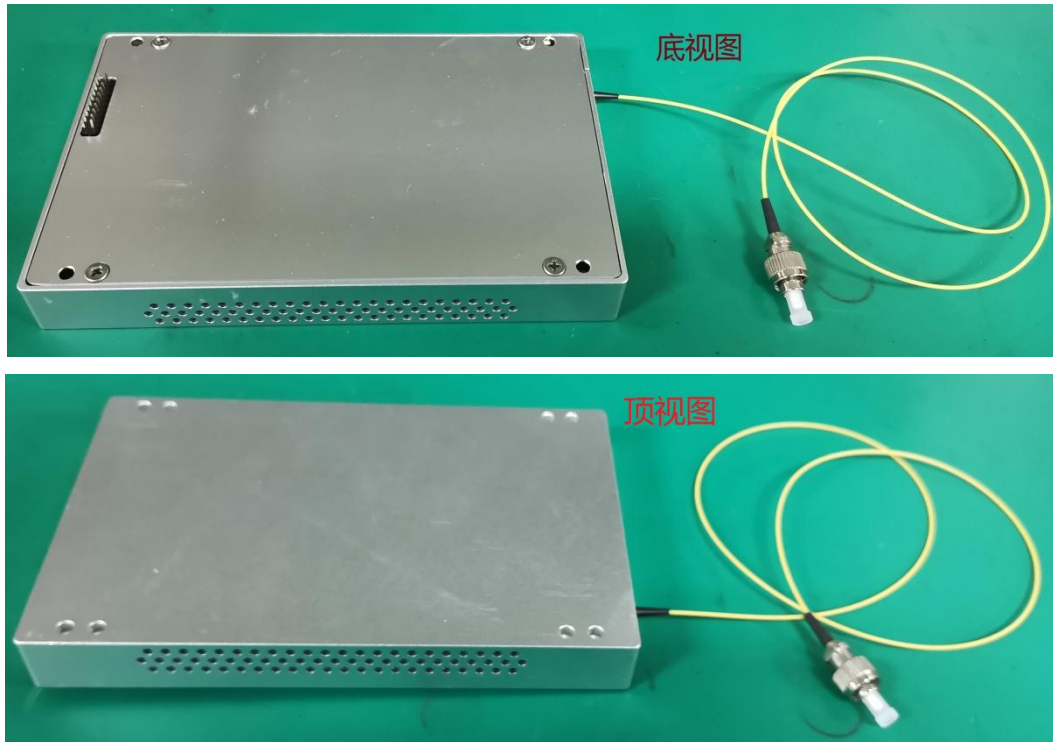


Figure 2: External view of the module

### 6.3 Product Interface Definitions

#### ● Optical Interface

The optical interfaces on the OTDR module all utilise FC/UPC fibre connectors (unless otherwise specified by the user), enabling the output of OTDR test optical pulse signals and the input coupling of reverse Rayleigh scattering and Fresnel reflection signals from the fibre.

#### ● Electrical Interface

The OTDR module offers two types of electrical interface for users to choose from: a  $2 \times 10$  (20P) dual-row pin header with a pitch of 1.27 mm, and a  $2 \times 10$  (20P) dual-row pin header with a pitch of 2 mm. Unless otherwise specified by the user, the default is the  $2 \times 10$  (20P) dual-row pin header with a pitch of 2 mm. The schematic diagram of the electrical interface is shown in Figure 3.

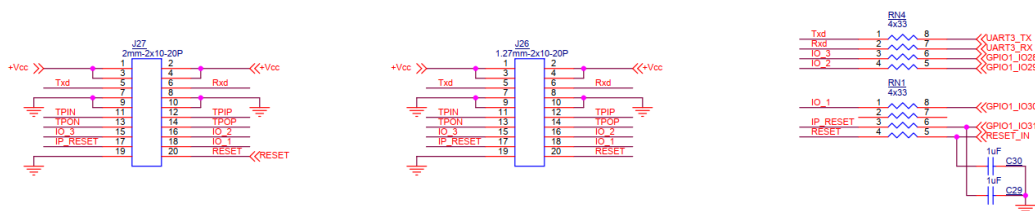


Figure 3: Schematic diagram of the electrical interface for the

The electrical interfaces primarily comprise the power supply inputs (+VCC and

power ground), an Ethernet interface, a serial port (spare) and other I/O interfaces (IP reset, external reset and spare I/O, etc.), enabling functions such as module power supply, network communication and other I/O control.

The signal definitions remain consistent across the two model specifications of the electrical interfaces (a 2×10 (20P) dual-row header with a pitch of 1.27 mm and a 2×10 (20P) dual-row header with a pitch of 2 mm); detailed pin definitions and specific descriptions are shown in Table 1.

Table 1: Electrical Interface Pin Definitions

| Pin No. | Name | Attribute | Type                  | Pin Definition Description                                                     |
|---------|------|-----------|-----------------------|--------------------------------------------------------------------------------|
| 1       | VCC  | P         | PS                    | DC power supply input, rated voltage 6V to 12V, voltage ripple less than 20mV. |
| 2       | VCC  | P         | PS                    |                                                                                |
| 3       | VCC  | P         | PS                    |                                                                                |
| 4       | VCC  | P         | PS                    |                                                                                |
| 5       | TxD  | O         | LVTTL                 | Serial transmission                                                            |
| 6       | RxD  | I         | LVTTL                 | Serial Receive                                                                 |
| 7       | GND  | P         | PS                    | Ground                                                                         |
| 8       | GND  | P         | PS                    | Ground                                                                         |
| 9       | GND  | P         | PS                    | Ground                                                                         |
| 10      | GND  | P         | PS                    | Ground                                                                         |
| 11      | TPIN | I         | Differential I signal | 10/100M Base-T data reception                                                  |
| 12      | TPIP | I         |                       |                                                                                |
| 13      | TPON | O         |                       | 10/100M Base-T data transmission                                               |

|                                                                                               |          |     |                         |                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------|----------|-----|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 14                                                                                            | TPOP     | O   | Differentia<br>I signal |                                                                                                                                                          |
| 15                                                                                            | IO_3     | I/O | LVTTL                   | Reserved I/O, undefined                                                                                                                                  |
| 16                                                                                            | IO_2     | I/O | LVTTL                   | Remains lit when the Ethernet LINK is active; otherwise, it is off.                                                                                      |
| 17                                                                                            | IP_RESET | I   | LVTTL                   | IP reset signal (internally pulled up); a low level remains valid for 3 seconds; after reset, the module reverts to the default IP address 192.168.1.249 |
| 18                                                                                            | IO_1     | I/O | LVTTL                   | Flashes during OTDR testing; otherwise, remains off.                                                                                                     |
| 19                                                                                            | GND      | P   | PS                      | Ground                                                                                                                                                   |
| 20                                                                                            | RESET    | I   | LVTTL                   | Module reset signal (internal pull-down); valid for 3 seconds at a high level                                                                            |
| P: Power   I: Input   O: Output   I/O: Input/Output   PS: Power or Ground   NC:<br>No connect |          |     |                         |                                                                                                                                                          |

#### 6.4 Instructions for Routine Operation

The standard operating procedures for the OTDR module are simple and easy to learn. The specific operating steps are as follows:

- 1) Before powering on the module, first connect the module's electrical interface to the 2×10 (20-pin) dual-row female connector on the dedicated interface board Portboard\_45dB\_V1 (or other device);

- 2) Use a network cable to connect the dedicated interface board Portboard\_45dB\_V1 (or other device) to the Ethernet port on a PC (an Ethernet switch may also be used as an intermediary);
- 3) Use an FC/FC flange (unless other specialised fibre optic connectors are required) to connect the module's fibre optic interface to the interface of the fibre under test ;
- 4) Power the module via the power supply port on the dedicated interface board Portboard\_45dB\_V1 (or other device). Refer to Figure 4 for a detailed schematic of the test connection for the dedicated interface board Portboard\_45dB\_V1;

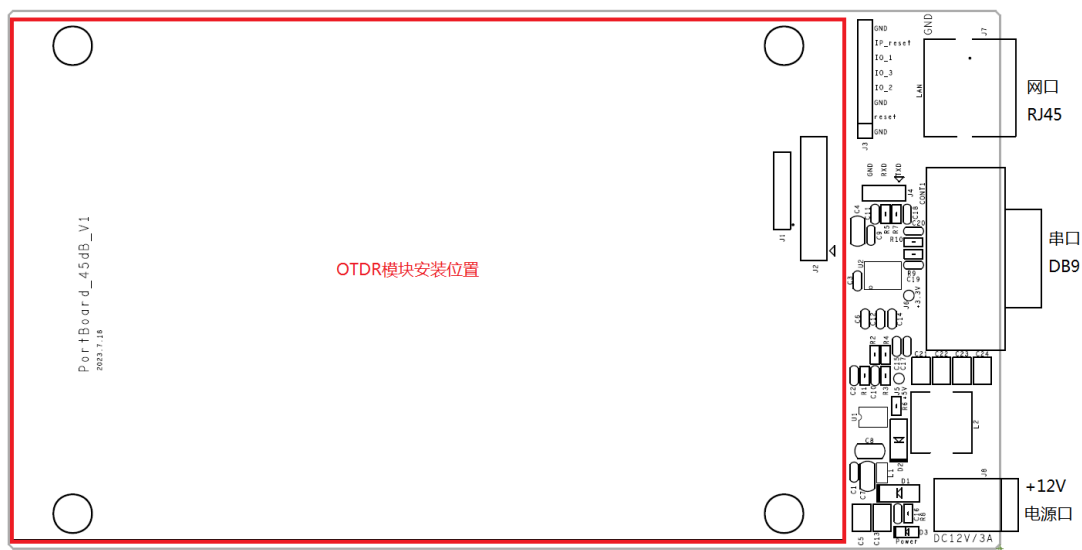


Figure 4: Test connection diagram for the interface board

- 5) Launch the module client software ‘ ’ (Mini OTDR Module Manager V3.1.3) on a PC, and connect to the module using its IP address (default: 192.168.1.249). A diagram illustrating the ‘Connect’ operation in the software is shown in Figure 5. Once the connection has been successfully established, follow the instructions in the ‘Module Client Software (V3.1.3) Manual’ to perform testing.

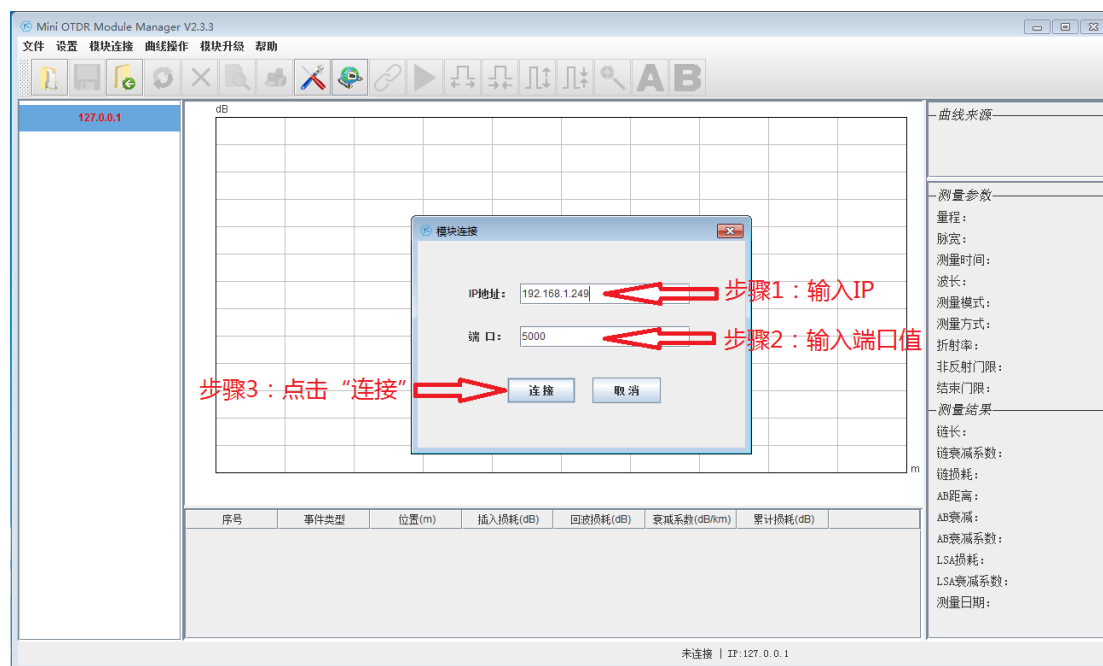


Figure 5: Diagram illustrating the software “Connect” operation

## 6.5 Operational Precautions

- 1) During module operation, operators should **take precautions against** electrostatic discharge to prevent damage to the module;
- 2) Pay attention to the alignment of the module’ s electrical interface with the 2×10 (20P) dual-row female connector on the dedicated interface board (or other equipment) to prevent damage to the module caused by incorrect pin signal connections;
- 3) Note that the IP settings for the PC’ s Ethernet interface must match those of the module ;
- 4) Note that when connecting the module’ s fibre optic interface to the fibre under test, the fibre connector types must be identical; otherwise, a compatible fibre patch cord must be used for the connection;
- 5) Before connecting the fibre optic interfaces, please ensure that the fibre end faces are cleaned to prevent abnormal OTDR test curves caused by poor fibre connections;
- 6) Before powering on the module, please ensure that **the polarity, voltage range and voltage ripple of the module’ s power supply** are correct to prevent damage to the module or a deterioration in its technical specifications;
- 7) When the module is operating continuously for extended periods, ensure that **the metal casing is cooled by blowing air** over it to prevent overheating, which could lead to a decline in product performance.

---

## 7. Technical Specifications

### 7.1 Naming Conventions

OTDR module product model naming conventions: Our proprietary products utilise a company-specific naming system, comprising a base model number followed by identifying symbols. For example, the model designation **M4745A-FU** is detailed as follows:

**M    47   45   A   –   FU**



(a) (b) (c) (d) (e)

|                        |                          |
|------------------------|--------------------------|
| Module model           | <b>M4745A-FU</b>         |
| Operating wavelength   | 1310/1550 ( $\pm 20$ nm) |
| Dynamic Range (SNR=1)  | 47/45 dB                 |
| Optical Interface Type | FC/UPC connector         |

Notes:

(a) Product base model

M — Basic model;

MT — Customised model, used to distinguish between different product types.

(b) Maximum value of the minimum wavelength dynamic range used by the product

(c) Maximum value of the larger wavelength dynamic range used by the product

Example 1: M4745A-FU — dual-wavelength model; 47 represents the maximum dynamic range at 1310, and 45 represents the maximum dynamic range at 1550;

Example 2: M4500C-FU — single-wavelength model; 45 denotes the maximum dynamic range at 1550 nm; 00 has no specific meaning; ; the optical interface type is an FC/UPC connector;

Example 3: M4300D-FU — single-wavelength model; 43 denotes the maximum dynamic range at 1625 nm; 00 has no specific meaning; the optical interface type is an FC/UPC connector;

Example 4: M4000D-FU — single-wavelength model; 40 denotes the maximum dynamic

range at 1625, whilst 00 has no specific meaning; the optical interface type is an FC/UPC connector;

Example 5: MT4000D-LU — customised single-wavelength model; ‘40’ denotes the maximum dynamic range at the 1625 wavelength; ‘00’ has no specific meaning; the optical interface type is an LC/UPC connector.

(d) Product operating wavelength code

A — Operating wavelengths are 1310/1550 nm (dual wavelength);

B — Operating wavelength is 1310 nm (single wavelength);

C — Operating wavelength: 1550 nm (single wavelength);

D — Single operating wavelength of 1625 nm;

E — Single operating wavelength of 1650 nm.

(e) Product optical interface type

Example 1: FU — The optical interface type is FC/UPC; the connector type is FC;  
the fibre end-face type is UPC;

Example 2: FA — The optical interface type is FC/APC; the connector type is FC;  
the fibre end-face type is APC;

Example 3: LU — The optical interface type is LC/UPC; the connector type is LC;  
the fibre end-face type is UPC.

In addition to the optical interface types listed above, users may customise optical interfaces according to their own requirements.

## 7.2 Technical Specifications

The OTDR modules supplied by our company are available in a range of models and specifications. The key technical specifications for the standard models are shown in Table 2; only a selection of models is listed, and we offer customisation for a wide range of models.

Table 2: Key Technical Specifications for General-Purpose Models of the

| Product Model                                                       | M4745A             | M4500C       | M4300D       | M4000D       | M3800D       |
|---------------------------------------------------------------------|--------------------|--------------|--------------|--------------|--------------|
| Dynamic range                                                       | $\geq 47$ dB/45 dB | $\geq 45$ dB | $\geq 43$ dB | $\geq 40$ dB | $\geq 38$ dB |
| Measurement parameters: 300 km/20 $\mu$ s/180 s, SNR = 1, at 25 ° C |                    |              |              |              |              |

|                                |                                                                                                              |                                    |                                    |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------|------------------------------------|
| Operating wavelength           | $1310 \pm 20 \text{ nm}$ & $1550 \pm 20 \text{ nm}$                                                          | $1550 \pm 20 \text{ nm}$           | $1625 \pm 20 \text{ nm}$           |
| Test range                     | 1, 5, 10, 30, 60, 100, 200, 300 km                                                                           |                                    |                                    |
| Pulse width                    | 5, 10, 20, 40, 80, 160, 320, 640, 1280, 2560, 5120, 10240, 20480 ns                                          |                                    |                                    |
| Event/Attenuation<br>Dead zone | $\leq 1.2 \text{ m}/6.0 \text{ m}$                                                                           | $\leq 1.2 \text{ m}/6.0 \text{ m}$ | $\leq 1.2 \text{ m}/7.0 \text{ m}$ |
|                                | Measurement parameters: 1 km/5 ns/15 s, at 25 ° C                                                            |                                    |                                    |
| Test distance<br>Uncertainty   | $\pm (0.75 + 5 \times 10^{-5} \times \text{distance} + \text{sampling resolution}) \text{ m}$                |                                    |                                    |
| Power supply                   | DC operating voltage: +6 V to 12 V; voltage ripple less than 20 mV                                           |                                    |                                    |
| Module power consumption       | $\leq 10 \text{ W}$                                                                                          |                                    |                                    |
| Number of sampling points      | 32,000                                                                                                       |                                    |                                    |
| Electrical interface           | 2×10 dual-row pin headers with 2 mm pitch (default) or 2×10 dual-row pin headers with 1.27 mm pitch          |                                    |                                    |
| Fibre-optic interface          | 9/125 μm single-mode fibre, FC/UPC connectors (unless otherwise specified by the user), length approx. 50 cm |                                    |                                    |
| Dimensions                     | Metal casing, length × width × height = 143 mm × 89 mm × 17 mm                                               |                                    |                                    |
| Operating environment          | Operating temperature: -5° C to +55° C                                                                       |                                    |                                    |
|                                | Storage temperature: -40° C to +85° C                                                                        |                                    |                                    |
|                                | Relative humidity: 5% to 85% (non-condensing)                                                                |                                    |                                    |

The relationship between the test range, pulse width and sampling accuracy of the OTDR modules supplied by our company ( ) is shown in Table 3 ( ).

**Table 3: Relationship between test range, pulse width and sampling accuracy**

| Test Range | Pulse width                                          | Sampling accuracy |
|------------|------------------------------------------------------|-------------------|
| 1 km       | 5, 10, 20, 40 ns                                     | 0.128 m           |
| 5 km       | 5, 10, 20, 40, 80, 160 ns                            | 0.128 m           |
| 10 km      | 5, 10, 20, 40, 80, 160, 320 ns                       | 0.255 m           |
| 30 km      | 5, 10, 20, 40, 80, 160, 320 ns                       | 0.51 m            |
| 60 km      | 40, 80, 160, 320, 640, 1280, 2560 ns                 | 1.02 m            |
| 100 km     | 80, 160, 320, 640, 1280, 2560, 5120, 10240 ns        | 2.04 m            |
| 200 km     | 80, 160, 320, 640, 1280, 2560, 5120, 10240, 20480 ns | 4.08 m            |
| 300 km     | 80, 160, 320, 640, 1280, 2560, 5120, 10240, 20480 ns | 8.16 m            |



---

### 7.3 User Customisation and Secondary Development Guidelines

The OTDR modules supplied by our company support customisation. Customisation options primarily include module model, operating wavelength, dynamic range and fibre interfaces. Should you require any customisation, please contact our relevant marketing staff directly.

The OTDR modules supplied by our company support secondary development by users. Users may carry out secondary development using the ‘ Ethernet Interface Communication Protocol ’ provided by our company. Should users have any requirements, please contact the relevant product technical staff directly.

## 8. Maintenance Procedures

### 8.1 Precautions for Product Use

OTDR modules are precision opto-electronic integrated modules; the following precautions must be observed during storage and use:

- 1) When storing the module for extended periods, ensure the storage environment is protected against dust and moisture;
- 2) If the module is not in use for an extended period, it is recommended that it be powered on and used at least once every six months;
- 3) Please take note of the operating environment for the module products and do not use the modules outside the specified operating conditions;
- 4) Operators should take precautions against electrostatic discharge when handling the module;
- 5) Before connecting the fibre optic interfaces, please ensure that the fibre optic interface end faces are cleaned;
- 6) Before powering on the module, please ensure you check the polarity and voltage range of the power supply;
- 7) When the module is operating continuously for extended periods, please ensure that the metal casing is cooled by blowing air over it;
- 8) When adapting the module for secondary development applications, ensure that the winding radius of the fibre optic patch cords is no less than 15 mm, whilst also taking into account the heat dissipation design of the module’s metal casing.

### 8.1 Troubleshooting Common Product Faults

Common fault symptoms and their solutions for OTDR modules during use are described as

---

follows:

- 1) Fault Symptom: **After power-up, the client software on the PC is unable to establish a connection with the module;**

Solution:

- a) After switching off the power, check whether the module' s electrical interfaces are correctly connected; if not, ensure they are correctly connected before switching the power back on and re-establishing the connection;
  - b) Check whether the network cable connected to the network interface is loose; if it is loose (or disconnected), reconnect the cable and attempt to establish the connection again;
  - c) Check whether the IP address entered in the client software is correct; if not, re-enter the correct IP address and reconnect;
  - d) Check whether the PC' s network IP settings match those of the module. If they do not, amend the PC' s network IP settings and then reconnect;
  - e) If you cannot recall the IP address set for the module, you may reset the IP address via the IP reset pin (**IP\_RESET**). The IP address will revert to the default value of 192.168.1.249; please then reconnect using this default IP address;
  - f) If none of the above solutions resolve the issue, please contact our technical support team for further investigation and resolution.
- 2) Symptom: **Abnormally high (or tailing) reflection peaks at the front end of the OTDR test curve;**

Solution:

- a) Check whether the module' s fibre optic interface is contaminated or has a poor connection. If contamination or a poor connection is found, clean the interface using a cleaning tool, reconnect it, and repeat the OTDR curve test;
- b) Check whether the fibre end faces of the module' s fibre interface and

- 
- the fibre under test match. If they do not match, use a fibre adapter patch cord to establish a compatible connection, then repeat the OTDR curve test;
- c) Check whether the pigtail section of the module's fibre optic interface is damaged or broken. If the pigtail is damaged or broken, replace it promptly (or return it to the manufacturer for replacement), and then repeat the OTDR curve test;
  - d) If none of the above measures resolve the issue, please contact our technical support team for further investigation, analysis and resolution.
- 3) Symptom: **The OTDR test curve cannot accurately identify the termination event;**
- Remedial measures:
- a) Check whether the fibre optic interface of the module is contaminated or has a poor connection. If contamination or a poor connection is present, clean the interface using a cleaning tool, reconnect it, and then repeat the OTDR curve test;
  - b) Check whether the OTDR measurement parameters of the module match the length of the fibre under test. If they do not match, adjust the OTDR measurement parameters (e. g. range, pulse width, measurement duration, end threshold, etc.), then repeat the OTDR curve test;
  - c) If the above solutions prove ineffective, please contact our technical support team for further investigation, analysis and resolution.

---

## Appendix 1: OTDR Module 4.0 Technical Development Documentation

### 第一章 Ethernet Interface Communication Protocol

The OTDR Module is an OTDR module that provides an Ethernet interface; the host computer communicates with the module using the TCP network protocol, although it can also be customised to use the UDP protocol. This approach requires a minimum number of connections; users need only provide a host computer CPU, along with the appropriate power supply and storage circuits, to carry out secondary development. It is assumed here that users are familiar with the TCP/IP network protocol.

#### 3.1 IP Address

The default IP address of the OTDR module is 192.168.1.249, and the communication port number is 5000. This IP address can be modified using the communication protocol commands described below.

#### 3.2 Network Byte Order

For multi-byte data, differences between processors may give rise to byte order issues. CPUs are generally classified into two types: big-endian and little-endian. The main difference between them lies in whether, for multi-byte data in memory, the high-byte data is stored at a low address or the high-byte data is stored at a high address. For example, for the data 0x1234, if its memory address is p, in a big-endian device—where the high-byte data is stored at a low address—the byte data stored at address p will be 0x12, and the byte stored at address p+1 will be 0x34, which is quite intuitive; for little-endian devices, the situation is exactly the opposite: the low-order byte is stored at a lower address, so the byte stored at address p will be 0x34, and the byte stored at address p+1 will be 0x12, which aligns with human thinking.

Network byte order specifies that, during network transmission, multi-byte data is transmitted with the high-order byte first, adhering to the ‘what you see is what you get’ principle. For big-endian devices, as the high-byte data is stored

---

first in memory—which already conforms to network byte order—a block of memory data can be sent directly using the ``send`` function without causing any issues, even if the data contains multi-byte values. Similarly, when receiving data, if the data conforms to network byte order, it can be received directly without any processing, and even if the data contains multi-byte values, this will not cause any problems. The situation is different for little-endian devices. If the data contains multi-byte data, it must be converted to network byte order using the ``htons`` macro (or function) or the ``htonl`` macro (or function) before transmission, and after reception, it must be converted back to local byte order using the ``ntohs`` macro (or function) or the ``ntohl`` macro (or function).

Given that little-endian devices are more commonly used, it is hereby stipulated that network byte order shall not be used; all data shall be sent directly to the network using the ``send`` function! If the user's CPU is a little-endian device, all data can be processed directly without any modification; however, if the user's CPU is a big-endian device, they will need to write functions or macros to convert local byte order to protocol byte order, as well as functions or macros to convert protocol byte order back to local byte order; floating-point data will require further processing. For conversion functions that transform integers and floating-point numbers from local byte order to network byte order, please refer to Chapter 4, 'Typical Development Process for Ethernet Interfaces'.

The following example illustrates the processing of multi-byte data and floating-point numbers in a big-endian device. For a 32-bit integer  $i32 = 0x12345678$ , it is stored in memory in the order 0x78, 0x56, 0x34, 0x12; therefore, the order in which it is transmitted remains 0x78, 0x56, 0x34, 0x12, and the receiving end also receives the data in this order; For a 32-bit floating-point number  $f32 = 1.4685$ , the corresponding integer representation is 0x3FBBF7CF. In memory, the bytes are stored in the order 0xCF, 0xF7, 0xBB, 0x3F; therefore, the transmission order remains 0xCF, 0xF7, 0xBB, 0x3F, and the receiving end also receives it in this order. If the receiving CPU uses little-endian format, it can be parsed directly; otherwise, the order must be reversed before it can be parsed.

---

### 3.3 Response Frame

For all commands sent from the host computer, the module sends a response frame upon receipt: if the command parameters are incorrect, a parameter error code is returned; if the command is valid and requires a response, the module returns the data immediately if possible; if the data cannot be returned immediately or if the command does not require a response, the module returns a status code indicating that the command has been successfully processed. The status codes are defined in the ‘Status Codes’ section below.

3.4 The basic data types that may be used in this communication protocol are declared as follows:

| Data Type Identifiers | Description                                                                          |
|-----------------------|--------------------------------------------------------------------------------------|
| String                | String, terminated by ‘\0’                                                           |
| Int8                  | 8-bit signed integer                                                                 |
| Int16                 | 16-bit signed integer                                                                |
| Int32                 | 32-bit signed integer                                                                |
| UInt8                 | 8-bit unsigned integer                                                               |
| UInt16                | 16-bit unsigned integer                                                              |
| UInt32                | 32-bit unsigned integer                                                              |
| Float32               | 32-bit single-precision floating-point number (compliant with the IEEE 754 standard) |

### 3.5 Communication Protocol Standard

Communication between the host computer and the module utilises a network communication protocol standard developed by our company, the structure of which is as follows:

|                              |                                     |                        |                          |                               |                            |                              |                             |
|------------------------------|-------------------------------------|------------------------|--------------------------|-------------------------------|----------------------------|------------------------------|-----------------------------|
| Start<br>frame 16<br>STPKSTR | Total<br>frame<br>length 4<br>PKLEN | Version<br>number<br>4 | Frame<br>type 4<br>PKTPY | Source<br>address<br>4<br>SRC | Destinat<br>ion<br>address | Sequence<br>number 4<br>PKID | Reserved<br>Byte 4<br>RSVD1 |
|------------------------------|-------------------------------------|------------------------|--------------------------|-------------------------------|----------------------------|------------------------------|-----------------------------|

|                              |                            |                     |  |  |          |  |                              |
|------------------------------|----------------------------|---------------------|--|--|----------|--|------------------------------|
|                              |                            | REV                 |  |  | 4<br>DST |  |                              |
| Command<br>Code 4<br>CMDCODE | Data<br>length 4<br>DATLEN | Data area n<br>DATA |  |  |          |  | Reserved<br>bytes 4<br>RSVD2 |

The following is a description of each field

- 1) Start Frame (STPKSTR): 16 bytes in length; identifies the start of a frame; of type String and terminated by ‘\0’ ; set here to GLink0tdr-3800M;
- 2) Total Frame Length (PKLEN): 4 bytes in length, of type Uint32, indicating the length of the entire data packet; its value is  $16 + 10 \times 4 + n$ ;
- 3) Version Number (REV): 4 bytes in length, of type Uint32, indicating the protocol version currently in use; for example, 0x00000001 indicates protocol version v0.001; not processed in this protocol;
- 4) Frame Type (PKTYP): 4 bytes in length, type Uint32. For example, 0x00000000 indicates a command frame sent from the host computer, whilst 0x00000001 indicates a command response frame sent from the module; this field is not processed in this protocol;
- 5) Source Address (SRC): 4 bytes in length, type Uint32; the sender enters its own IP address as a hexadecimal value, indicating the source of the data packet; not processed in this protocol; may be set to 0;
- 6) Destination Address (DST): 4 bytes in length, type Uint32. The sender enters the destination IP address to which the packet is sent, expressed as a hexadecimal value; this is not processed in this protocol and should be set to 0;
- 7) Sequence Number (PKID): 4 bytes in length, type Uint32; filled in by the sender, incremented sequentially each time; the receiver returns it directly without modification; used to uniquely identify a data packet; not processed in this protocol (TCP); may be set to any value; if the UDP protocol is used, this field

---

may be used to indicate the sequence number of a fragment.

- 8) Command Code (CMDCODE): 4 bytes in length, type Uint32, filled in by the sender, used to indicate an operational command. This protocol specifies that for commands sent from the host computer, the command code range is 0x10000000 to 0x7fffffff; for commands sent from the OTDR module, the command code range is 0x90000000 to 0xffffffff;
- 9) Data Length (DATLEN): 4 bytes in length, of type Uint32, indicating the length of the subsequent data section; its value is n;
- 10) Data Section (DATA): Length n bytes; data specific to the command in question; the data format is to be agreed separately for each specific project;
- 11) Reserved Bytes (RSVD1 - RSVD2): 4 bytes in length, of type Uint32; reserved for future expansion; filled with 0xffffffff where not used;

### 3.6 All Command Formats for the OTDR Module

The commands supported by the OTDR Module are listed below.

#### 3.6.1 Host Computer Test Parameter Settings

**Command ID:** 0x10000000

**Command description:** The command to set test parameters is used to configure the parameters required to initiate a test. The host computer must ensure the parameters are valid; if they are invalid, the module will return an error message or automatically correct certain parameters. Upon receiving a valid command, the module will immediately initiate the test.

**Command Data:** The various data fields are as follows

| Data Fields      | Description                                                                                                                                                                            | Data Type |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Measurement Mode | Set the OTDR mode, i.e. whether to use averaging mode or real-time mode.<br><br>1: Averaging mode<br>2: Real-time mode<br><br>0: Idle mode; no test is initiated<br><br>Other: Invalid | Uint32    |



|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |        |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Optimisation Mode | Retain                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Uint32 |
| Reserved          | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Uint32 |
| Enable refresh    | <p>Applies only to averaging mode; enables or disables data refresh</p> <p>0: Disable data refresh</p> <p>Other: Enable data refresh</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Uint32 |
| Refresh Period    | <p>Sets the refresh cycle for the averaging mode when data refresh is enabled, as well as the measurement time for real-time mode, in ms. This value is invalid when in averaging mode and data refresh is disabled. The refresh cycle must be greater than 1000 ms, must not exceed 5 s, and must be an integer multiple of 200 ms. If set to less than 1000, the OTDR module automatically adjusts it to 1000; if set to more than 5 s, the OTDR module automatically adjusts it to 5 s; if it is not a multiple of 200, the OTDR module automatically rounds up to the nearest multiple of 200; that is, if set to 700, the result is 1000; if set to 1050, the result is 1200; if set to 5500, the result is 5000.</p> <p>Calculation of refresh count: If the refresh interval is set to 2000 ms and the test duration is 15 s, then <math>15000/2000 = 7.5</math>. If there are 7 refreshes, taking a total of 14 s, only 1 s remains at the end. As this duration is shorter than a single refresh cycle, it may lead to real-time</p> | Uint32 |

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |        |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
|                 | <p>errors. Therefore, the OTDR refreshes only 6 times; the data uploaded during the final refresh is not refreshed data, but rather the data collected over the entire test duration. However, if the refresh interval is set to 1000 ms and the test duration is 15 s, then <math>15000/1000 = 15</math>, meaning there will be 14 refreshes, with the final upload comprising all the data.</p> <p>In other words: Number of refreshes = <math>(\text{int})(\text{test duration} / \text{refresh interval}) - 1</math></p> <p>Note: As the first refresh involves retrieving data, its time interval is longer than the refresh interval; subsequent intervals are equal to the refresh interval.</p> |        |
| Test Wavelength | <p>Set the current test wavelength, encoded in decimal. Depending on the hardware, the following wavelengths are supported: single-mode 1310, 1550, 1625, etc.; multimode 850, 1300. See notes for details.</p> <p>850: Wavelength is 850</p> <p>1300: Wavelength is 1300</p> <p>1310: Wavelength is 1310</p> <p>1550: Wavelength is 1550</p> <p>1625: Wavelength is 1625</p> <p>.....</p>                                                                                                                                                                                                                                                                                                              | UInt32 |
| Test range      | <p>Set the current test range, in metres. For example, to measure 30 km, the value should be 30,000. When the value is set to 0, this</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | UInt32 |

|                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |        |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
|                  | <p>indicates that automatic testing is being used; in this case, the test pulse width does not need to be set.</p> <p>Available range values are: 0, 1 km, 5 km, 10 km, 30 km, 60 km, 100 km, 180 km</p> <p>If the user sets a range length other than those listed, the OTDR module will adjust it upwards to the nearest supported range option. For example, if the user sets the range to 20,000 (indicating a desired range of 20 km), but the OTDR does not support this range, it will automatically adjust to 30 km.</p> |        |
| Test pulse width | <p>Sets the current test pulse width, in ns. For example, a pulse width of 2.56 <math>\mu</math>s corresponds to a value of 2560. When the test range is set to 0, automatic testing is used and this value does not need to be set.</p> <p>Pulse width values are: 10, 20, 40, 80, 160, 320, 640, 1280, 2560, 5120, 10240, 20480</p> <p>Pulse width must be associated with a specific range; not all pulse widths are permitted for every range. Please refer to the table below for details</p>                               | Uint32 |
| Measurement time | <p>Set the measurement time for the averaging mode, in ms. For example, a duration of 15 s corresponds to a value of 15,000. If the duration exceeds 180 s (3 min), it is truncated to 180 s; if it is less than 1,000 ms, it is truncated to 1,000 ms. The value</p>                                                                                                                                                                                                                                                            | Uint32 |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |         |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|                          | <p>must be greater than the refresh cycle and must be an integer multiple of 1,000. If the value is less than the refresh cycle, it is automatically adjusted to the refresh cycle; if it is not a multiple of 1000, it is rounded up to the nearest multiple of 1000. For example, if set to 1500, it is automatically changed to 2000</p> <p>If the OTDR measurement mode is set to real-time mode, this value has no effect. The refresh cycle for real-time mode—which also serves as its measurement time—uses the refresh cycle value specified above.</p> |         |
| Group Refractive Index   | Sets the group refractive index of the fibre; a 32-bit floating-point number is used, with a value between 1 and 2.                                                                                                                                                                                                                                                                                                                                                                                                                                              | Float32 |
| End threshold            | Expressed in dB. Affects the termination event point; a typical value is 5. In this OTDR module, it is defined such that when the curve drops to this value, subsequent event points are filtered out; it is not defined as the termination point of the fibre link when the insertion loss of a particular event exceeds this value.                                                                                                                                                                                                                            | Float32 |
| Non-reflection threshold | Unit: dB. Affects non-reflection event points. When set to 0, this indicates that the automatic non-reflection threshold is being used.                                                                                                                                                                                                                                                                                                                                                                                                                          | Float32 |

Appendix: Correspondence Table for Range and Pulse Width (Single-mode 1310, 1550, 1625)

| Range (km) | Available Pulse Widths (ns)    | Default pulse width for automatic testing (ns) |
|------------|--------------------------------|------------------------------------------------|
| 1          | 10, 20, 40, 80, 160            | 80                                             |
| 5          | 20, 40, 80, 160, 320           | 160                                            |
| 10         | 40, 80, 160, 320, 640          | 320                                            |
| 30         | 80, 160, 320, 640, 1280        | 640                                            |
| 60         | 160, 320, 640, 1280, 2560      | 1280                                           |
| 100        | 640, 1280, 2560, 5120, 10240   | 2560                                           |
| 180        | 1280, 2560, 5120, 10240, 20480 | 5120                                           |

Appendix: Correspondence Table for Range and Pulse Width (Multimode)

| Multimode 850  |                              |                                                |
|----------------|------------------------------|------------------------------------------------|
| Range (km)     | Available Pulse Widths (ns)  | Default Pulse Width for Automatic Testing (ns) |
| 1              | 20, 40, 80, 160              | 80                                             |
| 5              | 20, 40, 80, 160, 320         | 160                                            |
| 10             | 40, 80, 160, 320, 640        | 320                                            |
| 30             | 80, 160, 320, 640, 1280      | 640                                            |
| Multimode 1300 |                              |                                                |
| 1              | 20, 40, 80, 160              | 80                                             |
| 5              | 20, 40, 80, 160, 320         | 160                                            |
| 10             | 40, 80, 160, 320, 640        | 320                                            |
| 30             | 80, 160, 320, 640, 1280      | 640                                            |
| 60             | 160, 320, 640, 1280, 2560    | 1280                                           |
| 100            | 640, 1280, 2560, 5120, 10240 | 2560                                           |

Note: This command instructs the OTDR module to perform a test, but the OTDR module cannot return data immediately. Therefore, if the command parameters are invalid, an error message is returned; if the command is valid, a command processing success

---

code is returned. Once the refresh cycle is reached or the test is complete, the OTDR module uploads the data in the format specified in sections 3.6.103.6.11 and 11 below. Once the host computer has initiated a test, it should not send this command again whilst the test is in progress; instead, it should stop the test or restart it after completion. Failure to do so may result in data errors. Consequently, if the OTDR is currently conducting a test and receives a new test command, it will return the status code ‘Invalid Test Request’.

### 3.6.2 Host computer cancels test

**Command ID:** 0x10000001

**Command description:** The host computer cancels the current test.

**Command data:** The individual data fields are as follows

| Data Fields  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Data Type |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Control Mode | <p>Determines whether to cancel or terminate the current test.</p> <p>Cancelling the test terminates it completely, ending the test and entering idle mode, whilst discarding all data currently acquired;</p> <p>Terminating the test, on the other hand, stops the test and enters idle mode, whilst performing data processing based on the data currently acquired.</p> <p>1: Cancel test</p> <p>2: Terminate Test</p> <p>Other: No action; neither cancel nor terminate</p> | UInt32    |

Note: Whether the module returns success code 0 after stopping the test depends on the user’s selection. If ‘Terminate Test’ is selected, regardless of whether in averaging mode or real-time mode, the currently acquired data will be processed

---

once to calculate the event point, which will then be uploaded together with the test curve and test condition information; therefore, code 0 will not be returned at this point. If ‘Cancel Test’ is selected, code 0 will be returned once the OTDR module stops the test and enters idle mode. If any other value is selected, the OTDR module will not respond to the host computer.

During an automatic test, the system first spends 1 second learning the fibre link information. If the user selects ‘Cancel’ or ‘Abort’ during this learning process, the OTDR module will cancel the current test, return code 0, and will not return any other data.

### 3.6.3 IP Configuration Command

**Command ID: 0x10000002**

**Command description:** The host computer sets the OTDR module’ s IP address

**Command data:** The various data fields are as follows

| Data Fields     | Description                                                                                                 | Data Type      |
|-----------------|-------------------------------------------------------------------------------------------------------------|----------------|
| IP Address      | Contains the IP address to be set; use a string, such as “192.168.1.122”, ending with ‘\0’                  | String<br>[16] |
| Subnet mask     | Contains the subnet mask to be set; use a string, such as “255.255.255.0”, ending with ‘\0’                 | String<br>[16] |
| Default gateway | Contains the default gateway to be set, expressed as a string, such as “192.168.1.1” , and ending with ‘\0’ | String<br>[16] |

Note: Upon receiving the data, if the IP address is invalid, the OTDR module returns an ‘IP error’ code; if the data is valid, it returns a command processing success code after successfully updating the IP address and restarts the network. The host computer should reconnect 5 seconds after the restart.

### 3.6.4 Heartbeat commands between the host computer and the OTDR

**Command Identifier : 0x10000003**

---

**Command description:** The host computer sends this command to communicate with the OTDR as a heartbeat check

**Command Data:** This command requires no data; the data field is empty

Note: When the OTDR module receives the command, it returns an integer using command 0x90000002; the host computer need only receive this data and does not need to process it.

### 3.6.5 Host Computer Command to Query Measurement Status

**Command ID: 0x10000005**

**Command description:** The host computer uses this command to query the current test status of the OTDR module.

**Command data:** This command requires no data; the data field is empty

Note: The OTDR module uses command 0x90000005 to return a set of data representing its current status.

### 3.6.6 Host Command to Set the Timed Reset Period in Idle State

**Command ID: 0x1000000A**

**Command description:** After the host computer establishes a TCP connection with the OTDR module on port 5000 via the command ```, if no data is exchanged within 1 minute, the OTDR module automatically disconnects and resets; prior to establishing a TCP connection on port 5000 between the host computer and the OTDR module, the unit automatically resets once every hour; this interval is referred to as the timed reset cycle whilst idle. The host computer may use this command to set this reset cycle.

**Command data:** The data fields are as follows

| Data fields       | Description                                                  | Data Type |
|-------------------|--------------------------------------------------------------|-----------|
| Timed reset cycle | Reset cycle in minutes; minimum 1 min, maximum 0x2222222 min | Uint32    |

Note: Once the OTDR module has saved this cycle, it returns a command execution success status; the new cycle will take effect after the next reset.

### 3.6.7 Host computer reads OTDR software version

**Command ID: 0x11000000**

**Command description:** The host computer can use this command to read the OTDR software version number, as well as the compilation date and time and device description information.

**Command data:** This command requires no data; the data field is empty

Note: The OTDR module uses command 0x91000000 to return the corresponding data.



---

### 3.6.8 Curve Reception Response Command

**Command ID:** 0x10000004

**Command Description:** Upon successfully receiving a set of curve data, the host computer must use this command to respond to the OTDR module.

**Command data:** This command requires no data; the data field is empty

Note: The OTDR module employs an acknowledgement mechanism to upload data in order to accommodate relatively busy networks; that is, after uploading a single refresh of the data curve or the entire data curve, it will wait for the host computer to send this command; otherwise, it will not upload the next frame of the data curve. Therefore, when the host computer successfully receives a set of curve data, it must use this command to acknowledge the OTDR module. To ensure that the host computer can always respond to the OTDR module, the host computer may send this command once after receiving any data transmitted from the OTDR.

### 3.6.9 OTDR Online Upgrade Command

**Command ID:** 0x20000000

**Command Description:** The host computer performs an online upgrade of the OTDR software. The OTDR online upgrade file is a hexadecimal file; once the host computer has obtained this file, it must transmit the entire file in a single transmission. Furthermore, the host computer must ensure that the OTDR is currently idle and not performing any tests before proceeding with the upgrade.

**Command Data:** The various data fields are as follows

| Data Fields          | Description                                                                                                                                                                                                                                                   | Data Type |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Upgrade file content | <p>The upgrade file is a hexadecimal file; the host computer uses the fread function to read the file, then sends all the file data to the OTDR module in one go.</p> <p>If the file data received by the OTDR is incorrect, the error code “Upgrade file</p> | UInt8     |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |  |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
|  | <p>content is incorrect ” is returned; if correct, the code “OTDR upgrade started” is returned. If, during the upgrade process, the received upgrade file is found to be an invalid code file, or if the upgrade fails due to insufficient memory or other reasons, the error code “ OTDR upgrade failed ” is returned; if the file is valid and the upgrade is successful, the code “ OTDR upgrade completed” is returned.</p> <p>Once the upgrade is complete, the OTDR will automatically restart; the host computer can reconnect 5 seconds after the restart.</p> |  |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

#### 3.6.10 Upload all test data

**Command ID:** 0x90000000

**Command description:** The OTDR module uploads all test results

**Command data:** Test results consist of three parts: (a) test condition information, (b) test result data, and (c) event point information. The individual data fields are as follows

##### (a) Test condition information

| Data field         | Description                                                                                                                                                     | Data Type |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Sampling Frequency | <p>The sampling frequency actually used, encoded in decimal. Unit: Hz.</p> <p>Used to calculate the actual position corresponding to a sample point number.</p> | Uint32    |
| Test Range         | The test range currently in use. Unit: m.                                                                                                                       | Uint32    |
| Test pulse width   | The test pulse width currently in use. Unit: ns.                                                                                                                | Uint32    |
| Test               | The test wavelength currently in use. Unit: ns.                                                                                                                 | Uint32    |

---

|                          |                                                                    |         |
|--------------------------|--------------------------------------------------------------------|---------|
| wavelength               |                                                                    |         |
| Test duration            | The duration of the current test, in ms.                           | Uint32  |
| Group refractive index   | The group refractive index of the line under test.                 | Float32 |
| Fibre length             | The link length of the fibre currently under test                  | Float32 |
| Link loss                | Link loss of the fibre currently being tested                      | Float32 |
| Link attenuation         | Link attenuation of the fibre currently being tested               | Float32 |
| Non-reflection threshold | Non-reflection threshold currently used in the test, in dB.        | Float32 |
| End threshold            | The end threshold currently used for the test, expressed in dB.    | Float32 |
| Test Mode                | The test mode for the current test: 1 is averaging, 2 is real-time | Uint32  |
| Test Method              | Current test method: 0 is automatic, 1 is manual.                  | Uint32  |

**(b) Test result data**

| Data fields           | Description                                                                                                                                                                                                                                                          | Data type |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Number of data points | The number of data points obtained from the test, up to a maximum of 32,000                                                                                                                                                                                          | Uint32    |
| Value of data point 1 | The value of the first data point on the curve plus 5 dB, multiplied by 1,000 and rounded to the nearest integer. For example: 25,000 represents 20.000 dB.<br>Upon receipt, the host computer first divides by 1000.0 and then subtracts 5.000 to restore the value | Uint16    |
| ...                   | ...                                                                                                                                                                                                                                                                  | ...       |
| the value of          | The value of the last data point on the curve                                                                                                                                                                                                                        | Uint16    |

|              |                                                                                                                                                                                                                   |  |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| data point N | multiplied by 1000 and rounded to the nearest whole number. For example: 25000 represents 20.000 dB.<br><br>Upon receipt, the host computer first divides the value by 1000.0 and then subtracts 5.000 to restore |  |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

(c) Event point information

| Data field                                                         | Description                                                                                                                                            | Data type |
|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Number of event points                                             | The total number of event points obtained from the test. Maximum of 500                                                                                | Uint32    |
| Distance to Event Point 1                                          | The sequence number of Event Point 1 within the uploaded data.                                                                                         | Uint32    |
| Type of event point 1                                              | Type of Event 1<br>0: Start event<br>1: Reflection event<br>2: Non-reflection event<br>3: End event                                                    | Uint32    |
| Echo loss at event point 1                                         | Echo loss at event point 1. For non-reflection event points, this value need not be calculated                                                         | float     |
| Insertion loss at event point 1                                    | Insertion loss at event point 1. This value need not be calculated for the start and end points of the fibre.                                          | float     |
| Average fibre attenuation coefficient between event points 1 and 0 | Average fibre attenuation coefficient between Event Point 1 and the preceding event point. This value need not be calculated for the fibre start point | float     |
| Cumulative loss at Event Point 1                                   | Total link loss from Event Point 1 to the start of the link under test.                                                                                | float     |
| ...                                                                | ...                                                                                                                                                    | ...       |

|                                                                      |                                                                                                                                                        |        |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Distance to event point N                                            | The sequence number of event point N within the uploaded data.                                                                                         | Uint32 |
| Type of event point N                                                | Type of event N<br>0: Start event<br>1: Reflection event<br>2: Non-reflective event<br>3: End event                                                    | Uint32 |
| Echo loss at event point N                                           | Echo loss at event point N. For non-reflection event points, this value need not be calculated                                                         | float  |
| Insertion loss at event point N                                      | Insertion loss at event point N. This value need not be calculated for the start and end points of the fibre.                                          | float  |
| Average fibre attenuation coefficient between event points N and N-1 | Average fibre attenuation coefficient between event point N and the preceding event point. This value need not be calculated for the fibre start point | float  |
| Cumulative at event point N                                          | Total link loss from event point N to the start of the link under test.                                                                                | float  |

Note: The OTDR uses different sampling rates to collect data across different measurement ranges. The higher the sampling rate, the denser the sampling, and the closer the corresponding distances between points; in other words, the sampling rate corresponds to the point spacing. Assuming the sampling rate is  $F_s$  (Hz), the point spacing can be derived from the OTDR

positioning formula as  $dx = \frac{c/n}{2F_s} = \frac{c}{2nF_s}$ . Therefore, if the event point has an index of  $m$  in

the curve array, its corresponding actual distance is  $x = m \times dx = \frac{m \times c}{2nF_s}$ .

The data for the curve calculated by the OTDR is of type 'float', with a minimum value of -5 and a maximum value not exceeding 50. In the upload process specified by this protocol, the curve values are first increased by 5 to ensure they are all non-negative, allowing them to be

---

represented as unsigned integer data. The values, after being increased by 5, are then multiplied by 1000, rounded to the nearest integer, and uploaded as `UInt16` type. This provides 3-digit precision, resulting in a smoother and more detailed curve. Upon receiving the curve data, the host computer should first divide it by 1000.0 and then subtract 5 to restore the data to the format used internally by the OTDR.

Event points generally have five attributes: position, type, insertion loss, return loss and cumulative loss (total loss); fibre segments generally have two attributes: fibre segment length and average attenuation coefficient. Under this protocol, the attributes of the fibre segment are merged into the event point attributes for upload. As the length of the fibre segment is defined as the distance between two adjacent event points—a value that can be expressed through the event point attributes and is therefore redundant—the fibre segment length is not uploaded. Instead, only the average attenuation coefficient of the fibre segment is uploaded, and this is included in the event point corresponding to the end of the fibre segment. Consequently, an event point has six parameters: location, type, insertion loss, return loss, cumulative loss (total loss), and the average attenuation coefficient of the fibre segment corresponding to the previous event point.

Of the six event point parameters uploaded as described above, in some cases certain parameters cannot be calculated or do not need to be calculated. Firstly, both position and type can be calculated, so they will always have reliable values; secondly, with regard to insertion loss, the values for the start and end events cannot be calculated, and if two events are too close together, insertion loss cannot be calculated either; as for return loss, only reflection events possess this attribute, whilst non-reflection events do not; As regards cumulative loss, the start event does not require calculation, and two event points that are too close together cannot be calculated; even if a value is derived, it is unreliable; as for the average attenuation coefficient of the fibre segment, it can be calculated if the fibre segment is sufficiently long, but if two event points are too close together—resulting in a fibre segment that is too short—the average attenuation coefficient cannot be calculated; even if a value is derived, it is not credible.

For all the attribute parameters mentioned above that cannot be calculated or do not need to be calculated, this protocol uniformly uses the reserved value 8192.0 to represent them; if the host computer detects that a particular attribute has this reserved value, it may apply special handling.

Furthermore, upon receiving this command, the host computer must respond to the module with a ‘curve reception response command’.

### 3.6.11 Uploading refreshed or real-time test data

Command identifier: 0x90000001

Command description: The OTDR module uploads refresh or real-time test results

Command data: The test results consist of only one part, namely the test result data, which corresponds to part (b) of the previous command “Upload All Test Data”. This is because refresh or real-time testing does not search for event points but only displays the test curve; consequently, there is no event point information. The various data fields are as follows

Test result data

| Data Fields               | Description                                                                                                                                                                                                                                                                          | Data Type |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Number of data points     | The total number of data points obtained from the test                                                                                                                                                                                                                               | Uint32    |
| Value of data point 1     | <p>The value of the first data point on the curve plus 5 dB, multiplied by 1,000 and rounded to the nearest whole number. For example: 25,000 represents 20.000 dB.</p> <p>Upon receipt, the host computer first divides by 1000.0 and then subtracts 5.000 to restore the value</p> | Uint16    |
| ...                       | ...                                                                                                                                                                                                                                                                                  | ...       |
| the value of data point N | <p>The value of the last data point on the curve multiplied by 1000 and rounded to the nearest whole number. For example: 25000 represents 20.000 dB.</p> <p>Upon receipt, the host computer first divides the value by 1000.0 and then subtracts 5.000 to restore</p>               | Uint16    |

Note: The minimum value of the curve data calculated by the OTDR and used in subsequent processing is -5; the maximum value depends on the pulse width and is generally less than 50. Therefore, to enable representation using unsigned Uint16 data during upload, a value of 5 must be manually added to ensure the value is non-negative, followed by multiplication by 1000 and rounding to the nearest integer. Upon receiving this curve data, the host computer should first divide by 1000 and then subtract 5; only then will the resulting curve match the OTDR’s processed output. Furthermore, upon receiving this command, the host computer must respond to the module using the ‘Curve Receive Response Command’.

### 3.6.12 Return Heartbeat Packet

**Command ID: 0x90000002**

**Command Description:** The OTDR module responds to the host computer's heartbeat packet

**Command Data:** The various data fields are as follows

| Data Fields      | Description                                                 | Data Type |
|------------------|-------------------------------------------------------------|-----------|
| Heartbeat status | Reserved value; no processing required by the host computer | UInt32    |

Note: This is a response to heartbeat command 0x10000003.

### 3.6.13 Upload test status information

**Command ID: 0x90000005**

**Command description:** The OTDR module returns the current test status

**Command data:** The individual data fields are as follows

| Data fields         | Description                                                                                                                                                          | Data Type |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Operating Mode      | The current operating mode of the OTDR module. If the mode is 'Idle', the following data is invalid<br>0: Idle, not tested<br>1: Averaging mode<br>2: Real-time mode | UInt32    |
| Wavelength          | Current test wavelength, in nm                                                                                                                                       | UInt32    |
| Range               | Current test range, in metres                                                                                                                                        | UInt32    |
| Pulse width         | Currently testing the pulse width of , in ns                                                                                                                         | UInt32    |
| Test duration       | Current test duration, in ms                                                                                                                                         | UInt32    |
| Time elapsed so far | If the mode is 'Average', this represents the total test time completed so far and can be used as a progress indicator                                               | UInt32    |

Note: When the measurement mode is 1 or 2, this indicates that measurement is in progress; when it is 0, it indicates that the test has not started or has been completed; this is the response to query command 0x10000005.

### 3.6.14 Upload version information

**Command ID: 0x91000000**

**Command description:** The OTDR module returns software version information

**Command data:** The individual data fields are as follows

| Data fields | Description | Data Type |
|-------------|-------------|-----------|
|-------------|-------------|-----------|



|                                 |                                                                                                                                                                          |            |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Major and minor version numbers | The upper 8 bits represent the major version number, and the lower 8 bits represent the minor version number                                                             | Uint16     |
| Compilation date: Year          |                                                                                                                                                                          | Uint16     |
| Compilation date: Month         | Value range: 1 - 12                                                                                                                                                      | Uint16     |
| Compilation date: day           | Minimum value is 1                                                                                                                                                       | Uint16     |
| Compilation time                | The upper 16 bits represent the hour (0 - 23); of the lower 16 bits, the upper 8 bits represent the minutes (0 - 59) and the lower 8 bits represent the seconds (0 - 59) | Uint32     |
| Device description              | A string describing the name of the current OTDR device, e.g. "GL3800M"                                                                                                  | String[16] |

Note: This is the response to query command 0x11000000.

### 3.6.15 Response Code

**Command ID:** 0xA0000000

**Command description:** The module returns a status code indicating the result of the current command

**Command data:** The status codes are as follows

| Status code                                             | Description                                                                                 |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------|
| 0                                                       | Command processed successfully                                                              |
| 1                                                       | Invalid frame start string                                                                  |
| 2                                                       | Invalid version number                                                                      |
| 3                                                       | Invalid frame type                                                                          |
| 4                                                       | Invalid command identifier                                                                  |
| 5                                                       | Invalid data length                                                                         |
| 6 - 15                                                  | Reserved                                                                                    |
| The following status codes are defined by this protocol |                                                                                             |
| 16                                                      | Invalid range or pulse width. A specific range imposes certain restrictions on pulse width. |
| 17                                                      | Group refractive index is invalid. The group refractive index must be between 1 and 2.      |

---

|                            |                                  |
|----------------------------|----------------------------------|
| 18                         | Non-reflection threshold invalid |
| 19                         | Invalid request test             |
| 20                         | Invalid IP address               |
| Online upgrade status code |                                  |
| 100                        | Error in upgrade file contents   |
| 101                        | OTDR has started the upgrade     |
| 102                        | OTDR upgrade failed              |
| 103                        | OTDR upgrade completed           |

## 第二章 OTDR Heartbeat Detection and Automatic Reset

Given that prolonged operation of the OTDR module may lead to issues caused by complex external network environments, a heartbeat detection and automatic reset mechanism has been specifically incorporated.

As the OTDR module communicates with the host software via the TCP protocol—which is a connection-oriented protocol—the OTDR module can detect whether the host software has established a connection with it. If a connection is established, the OTDR module requires the host software to send heartbeat packets at regular intervals; if no heartbeat packets are received within one minute, the OTDR module will deem the communication to be abnormal, disconnect the TCP connection and automatically reset. If, however, the host software has not established a connection with the OTDR module, the OTDR module will be unable to receive any heartbeat packet data and will automatically reset after one hour.

Furthermore, should any form of data transmission error occur, the OTDR module will automatically reset to rectify the error.

The host software and the OTDR module can exchange heartbeat data via three methods: test commands, the echo server, and the daytime server.

- As long as there is any form of data exchange between the host software and the OTDR—including test commands, stop-test commands, etc.—the OTDR

---

module will consider communication to be normal and will not disconnect and reset after one minute. Therefore, the host software can periodically send commands to the OTDR module, such as the ‘Receive Curve Response’ command, or invalid commands (provided they conform to the command format; ‘invalid’ here simply means that the command code is not supported).

- The OTDR module has a TCP port open on port 5555, known as the echo server. The host computer software can connect to the echo server via this port and send data of up to 64 bytes; upon receiving this data, the OTDR will return it to the host computer exactly as received. As the OTDR acts as a server, it does not actively terminate the TCP connection; therefore, once the connection has been established, the host computer software can carry out periodic communication without needing to reconnect. As this process involves data exchange, the OTDR will recognise the communication as normal and will not disconnect and reset after one minute.
- The OTDR module also enables TCP port 6666, known as the ‘daytime’ server. The host software can connect to the daytime server via this port, whereupon the OTDR immediately sends a string of no more than 16 bytes to the host software (in fact, this string represents the number of milliseconds elapsed since the OTDR was powered on) and immediately terminates the TCP connection. Consequently, once the connection is established, the host software must not send any data but should immediately receive the data (which may be discarded) and then terminate the connection. As this process involves data exchange, the OTDR will interpret the communication as normal and delay the reset until one minute later.

The host software may select any form of heartbeat data exchange. If Method 1 is chosen, the advantage is that no additional communication ports need to be opened, and the software workload is minimal; however, the synchronisation logic for commands must be handled correctly—for example, heartbeat commands may be omitted during testing or whilst the software is being updated. If Method 2 or Method 3 is

---

selected, the advantage is that the current host computer software does not need to be modified; the heartbeat exchange can be accomplished using the PC's operating system alone, i.e. by writing a simple PC-side programme whose function is to send data to the OTDR's echo server or daytime server at regular intervals; the disadvantage is that the host computer software is not involved in the heartbeat exchange and cannot monitor the status of the OTDR module (it cannot detect whether the OTDR module has gone offline due to a power failure or other reasons).

### 第三章 Typical Development Process for Ethernet Interfaces

#### 5.1 Overview

The OTDR module supports two test modes: averaging mode and real-time mode. Averaging mode is a mode in which the host computer initiates the test by sending a test command, and the OTDR module performs a single test on the fibre link according to the test parameters set by the user. In this mode, a single test command can only initiate one test. As the test duration is relatively long, the host computer may choose whether to enable data refresh during the test. If data refresh is enabled, the OTDR module will calculate a curve and upload it to the host computer once the test duration reaches the refresh cycle; the host computer should ensure it receives and displays the data in a timely manner. If data refresh is not enabled, the OTDR module will only calculate the curve and upload it once the test has concluded. Real-time mode is initiated by the host computer sending a test command. The OTDR module monitors the fibre-optic link in real time according to the user-defined test parameters and, upon reaching the monitoring interval, calculates and uploads the curve. The test does not end at this point but continues using the original test parameters to initiate further tests. In this mode, a single test command is sufficient to monitor the link in real time. The monitoring interval is generally not too long—typically 1 second—so data refresh is not used, or rather, data is constantly being refreshed.

The short-term averaging mode cannot be used as a substitute for real-time mode by repeatedly initiating tests, as the averaging mode takes a considerable amount

---

of time to calculate fibre information after the test is complete, which inevitably affects the refresh rate. In contrast, real-time mode does not calculate fibre information, and therefore offers a higher refresh rate.

Furthermore, the OTDR module supports both automatic and manual test modes. During an automatic test, the OTDR module uses specific parameters to test the fibre link for one second to characterise it; it then estimates the fibre length based on the data obtained and automatically adjusts the range and pulse width. Consequently, there is no need for the host computer to specify the range and pulse width. However, if the fibre is too long—for example, exceeding 100 km—the automatic test is likely to fail. Manual testing, on the other hand, requires the host computer to specify both the range and pulse width simultaneously. Naturally, the first frame uploaded during automatic testing will be slower than that during manual testing.

If refresh is enabled in average mode, the OTDR module will calculate the curve and upload it once the refresh cycle is reached; however, the upload cycle is often longer than the refresh cycle and is not a fixed value. This is because the upload cycle includes the overhead involved in the OTDR module reading sample data, the computational overhead, and transmission overhead caused by network congestion and other factors.

The uploaded data includes refresh data and final data from the averaging mode, as well as refresh data from the real-time mode and final data from the real-time mode after the test has been terminated. Of these four types of data, the refresh data from the real-time mode is identical to that from the averaging mode, and the final data from the real-time mode after the test has been terminated is also identical to the final data from the averaging mode; therefore, there are essentially only two types of data: refresh data and final data. Refresh data contains only curve values; it does not include test condition information or event point information, serving merely as a temporary display. Final data, on the other hand, includes test condition information, curve values and event point information, and constitutes the complete set of information regarding the fibre link test.

---

The length of the refresh data uploaded by the OTDR Module is fixed, whilst the length of the final data can be estimated. If the refractive index of the fibre is  $n$ , the sampling rate is  $F_s$ , and the speed of light is  $C$ , then the number of sampling points per metre is  $\text{Points}_{1m} = 2n * F_s / C$ . Therefore, for a measurement range of  $L$  km, the number of sampling points is  $\text{SampleNum} = (\text{Int}32)(1000 * L * \text{Points}_{1m}) + 1$ . Taking alignment issues into account, if  $\text{SampleNum}$  is an odd number, the actual number of sample points uploaded is  $\text{UploadLen} = \text{SampleNum} + 1$ ; if it is an even number, the actual number of sample points uploaded is  $\text{UploadLen} = \text{SampleNum}$ , with a maximum of 32,000. The OTDR Module uses a sampling rate of 400 MHz for measurement ranges of 1 km and 5 km, 200 MHz for a range of 10 km, and 100 MHz for a range of 30 km. at a range of 60 km, the sampling rate is 50 MHz; at a range of 100 km, the sampling rate is 25 MHz; and at a range of 180 km, the sampling rate is 12.5 MHz. Furthermore, the value used for the speed of light is 299792458, rather than  $3 \times 10^8$ . Consequently, the refresh packet size is  $\text{UploadLen} * \text{sizeof}(\text{Uint16}) + \text{frame overhead} = 2 * \text{UploadLen} + 60$  bytes; The final packet size is  $\text{UploadLen} * \text{sizeof}(\text{Uint16}) + \text{event point overhead} + \text{frame overhead} = 2 * \text{UploadLen} + 24 * \text{EventNum} + 116$  bytes`, where `EventNum` is the number of event points; the OTDR module can accommodate a maximum of 500 event points.

## 5.2 Development Process

Once the host computer has established a connection with the OTDR module, it uses command **0x10000000** to set the test parameters in accordance with the frame structure described in Chapter 2 and sends them to the OTDR module. If the OTDR module verifies that the command parameters are valid, it will initiate the test. As the OTDR module employs an active data upload method, the host computer must be ready at all times to receive and display the data, and to respond promptly to the OTDR module. The refresh data uploaded by the OTDR module does not contain test information or event point information; it contains only curve values, and the size of this data packet is fixed for a given range. The final data must also include test information and event point information, so the data packet is larger.

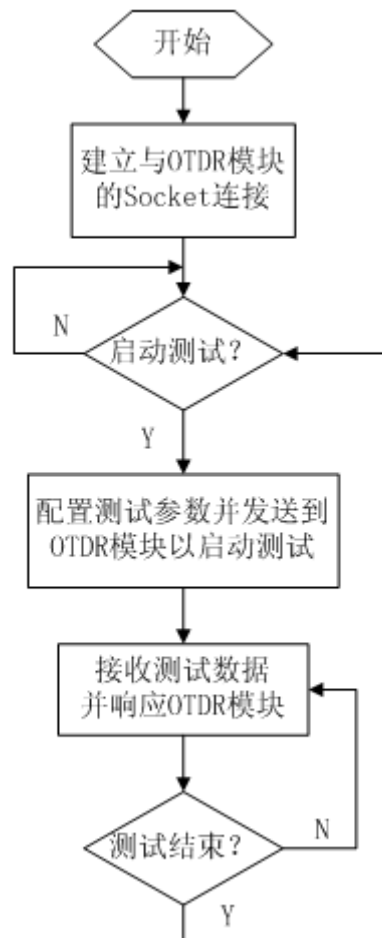
Once the test has commenced, the host computer must not send the test command

---

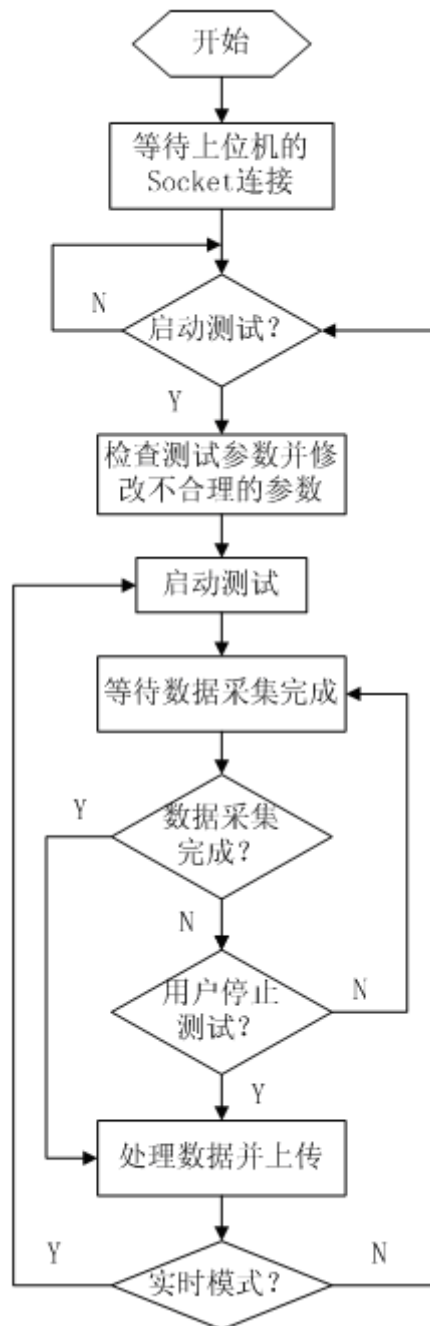
(0x10000000) again until the test has concluded. Failure to comply with this may result in a test error; consequently, the OTDR module will not process the 0x10000000 command whilst the test is in progress. If, during the test, the host computer wishes to discontinue the test, it should use **command 0x10000001** to cancel or stop the test; only then can a new test be initiated. Upon receiving the test cancellation command, if the OTDR module has already completed data acquisition and is processing the data, it will wait until data processing is complete before responding to further commands from the host computer.

If the SOCKET connection is terminated, or if a SOCKET transmission error occurs, the OTDR module will immediately cancel the current test and discard all acquired data.

Below is a diagram illustrating the host computer's development workflow.



The following is a diagram illustrating the OTDR module's operational workflow:



### 5.3 Development Programme (C language / Linux system)

The code below demonstrates how to initiate a test and receive data using the



---

C language on a Linux system.

First, the header file, which defines various constants and data structures:

```
// tcp-otdr.h

// Access OTDR using TCP

#ifndef _TCP_OTDR_H
#define _TCP_OTDR_H

#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <strings.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <sys/time.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <errno.h>

/*****                                CMD                                Macro
*****/

#define CMD_HOST_START_MEASURE      0x10000000    // Configure test parameters
and start the test

#define CMD_HOST_STOP_MEASURE       0x10000001    // Cancel or terminate the
test

#define CMD_HOST_SET_IP             0x10000002    // Set IP
#define CMD_HOST_NETWORK_IDLE       0x10000004    // Network idle
#define CMD_OTDR_UPDATE             0x20000000    // OTDR online update
#define CMD_DSP_UPLOAD_ALL_DATA     0x90000000    // Upload all test data
#define CMD_DSP_UPLOAD_REF_DATA     0x90000001    // Upload reference test
data

#define CMD_RESPONSE_STATE          0xA0000000    // Return status code
```

---

```

/*****                               Frame       Format       Definitions
*****/

// Version number

#define REV_ID      0

#define RSVD_VALUE  0xffffeeee

// Frame type

#define FRAMETYPE_HOST2TARGET  0   // Host command type
#define FRAMETYPE_TARGET2HOST  1   // Response command type

#define FRAME_SYNC_STRING  "GLinkOtdr-3800M"

// Frame header identifier

typedef struct
{
    char    FrameSync[16]; // GLinkOtdr-3800M

    uint32_t TotalLength;   // Total frame length

    uint32_t Rev;           // Version number

    uint32_t FrameType;     // Frame type

    uint32_t Src;           // Source address

    uint32_t Dst;           // Destination address

    uint32_t PacketID;      // Sequence number

    uint32_t RSVD;          // Reserved

}frame_header_t;

// Configure test parameters and start the test

typedef struct
{
    uint32_t cmd;           // Command code 0x10000000

    uint32_t len;           // Data length

    // Control section

    struct
    {
        uint32_t OtdrMode;           // OTDR mode

        uint32_t OtdrOptMode;        // OTDR optimisation mode

        uint32_t RSVD;
    }

```

---

```

        uint32_t  EnableRefresh;        // Enable or disable refresh

        uint32_t  RefreshPeriod_ms;    // Refresh period, in ms; must not be less
than 800

    }Ctrl;

    // Status section

    struct

    {
        uint32_t    Lambda_nm;          // Operating wavelength, in nm
        uint32_t    MeasureLength_m;     // Measurement length, in metres
        uint32_t    PulseWidth_ns;       // Light pulse width, in ns
        uint32_t    MeasureTime_ms;       // Measurement time, in ms
        float       n;                   // Refractive index
        float       EndThreshold;         // End threshold
        float       NonReflectThreshold; // Non-reflection threshold
    }State;

        uint32_t  RSVD;
    }start_measure_t;

// Cancel test

typedef struct

{
    uint32_t  cmd; // Command code 0x10000001
    uint32_t  len; // Data length

    // Control mode

    uint32_t  Cancel_Or_Abort; // Cancel or abort the test

    uint32_t  RSVD;
}OTDR_Cancel_t;

// IP address, subnet mask and default gateway

typedef struct

{
    uint32_t  cmd; // Command code 0x10000002
    uint32_t  len; // Data length

    char      LocalIPAddr[16];
    char      LocalIPMask[16];

```

---

```

    char    GatewayIP[16];

    uint32_t  RSVD;
}OTDR_IP_t;

// OTDR uploads all test data

#define      DATA_LEN          32000

#define      MAX_EVENT_NUM      500

#define RSVD_FLOAT          8192.0    // Float reserved value, indicating that
the value is indeterminate or need not be determined

typedef struct
{
    uint32_t  cmd;                // Command code 0x90000000

    uint32_t  len;               // Data length

    // Test condition information

    struct
    {
        uint32_t    SampleRate_Hz;

        uint32_t    MeasureLength_m;

        uint32_t    PulseWidth_ns;

        uint32_t    Lambda_nm;

        uint32_t    MeasureTime_ms;

        float    n;

        float    FiberLength;        // Fibre chain length

        float    FiberLoss;          // Link loss

        float    FiberAttenCoef;     // Link attenuation coefficient

        float    NonReflectThreshold; // Non-reflection threshold

        float    EndThreshold;       // End threshold

        uint32_t  OtdrMode;          // OTDR mode: real-time, average

        uint32_t  MeasureMode;       // Measurement mode: automatic, manual
    }MeasureParam;

    // Test data information

    struct
    {
        uint32_t    DataNum;

```

---

```

        uint16_t    dB_x1000[DATA_LEN];

    }OtdrData;

    // Event point information unit

    struct

    {
        uint32_t    EventNum;           // Number of event points

        // Event point

        struct

        {
            uint32_t    EventXlabel;     // The index of the event point in the
uploaded data array

            uint32_t    EventType;       // Event point type

            float    EventReflectLoss;   // Reflection loss / echo loss

            float    EventInsertLoss;    // Insertion loss

            float    AttenCoef;         // Attenuation coefficient between this
event and the next

            float    EventTotalLoss;     // Cumulative loss across event
points

        }EventPoint[MAX_EVENT_NUM];

    }Event;

    uint32_t    RSVD;

}otdr_result_t;

// Upload and refresh test data to the OTDR

typedef struct

{
    uint32_t    cmd;                   // Command code 0x90000001

    uint32_t    len;   // Data length

    // Test data information

    struct

    {
        uint32_t    DataNum;

        uint16_t    dB_x1000[DATA_LEN];

    }OtdrData;

    uint32_t    RSVD;

```

---

```

}otdr_ref_t;

// OTDR return frame command data structure

#define STATE_CODE_CMD_OK 0 // Command processed
successfully

#define STATE_CODE_FRAME_SYNC_ERROR 1 // Invalid frame start
string

#define STATE_CODE_REV_ERROR 2 // Invalid version
number

#define STATE_CODE_FRAME_TYPE_ERROR 3 // Invalid frame type

#define STATE_CODE_CMD_ID_ERROR 4 // Invalid command ID

#define STATE_CODE_PACKET_LENGTH_ERROR 5 // Invalid data length

#define STATE_CODE_ML_OR_PW_ERROR 16 // Invalid measurement
range or pulse width

#define STATE_CODE_N_ERROR 17 // Illegal group
refractive index

#define STATE_CODE_NR_ERROR 18 // Non-reflection
threshold invalid

#define STATE_CODE_OTDR_BUSY_MEASURE 19 // Invalid request for
measurement

#define STATE_CODE_IP_ERROR 20 // Invalid IP address

// Online upgrade status codes

#define STATE_CODE_FILE_CONTENT_ERROR 100 // Error in update file
content

#define STATE_CODE_OTDR_UPDATE_START 101 // Update started

#define STATE_CODE_OTDR_UPDATE_FAIL 102 // Update failed

#define STATE_CODE_OTDR_UPDATE_DONE 103 // Update complete

typedef struct
{
    uint32_t cmd; // Command code 0xA0000000

    uint32_t len; // Data length

    uint32_t StateCode; // Response code

```

---

```

        uint32_t  RSVD;
    }otdr_state_t;
#endif // _TCP_OTDR_H

    Next is the main file, which starts the test and receives data:

// tcp-otdr.c
// Access OTDR using TCP
#include "tcp-otdr.h"
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define      BUF_SIZE      (128*1024)
char buf[BUF_SIZE];      // Allocate a large buffer to hold data

// Convert host float to network float
float htonf(float t)
{
    uint32_t    ux;

    float        x = t;

    ux = *(uint32_t*)&x;

    ux = htonl(ux);

    return *(float*)&ux;
}

// Convert network float to host float
float ntohf(float t)
{
    uint32_t    ux;

    float        x = t;

    ux = *(uint32_t*)&x;

    ux = ntohl(ux);

    return *(float*)&ux;
}

// Response OTDR

int NetworkIdle(int s)

```

---

```

{   frame_header_t  *header;

    start_measure_t   *start;

    uint32_t tlen;

    tlen = sizeof(frame_header_t) + 12;

    header = (frame_header_t *)buf;

    strcpy(header->FrameSync, FRAME_SYNC_STRING);

    header->TotalLength = tlen;

    start = (start_measure_t *) (buf + sizeof(frame_header_t));

    start->cmd = CMD_HOST_NETWORK_IDLE;

    start->len = 0;

    return send(s, buf, tlen, 0);

}

// Set parameters and start the measurement

int HostStartMeasure(int s, int measurelen, int pw)

{   frame_header_t  *header;

    start_measure_t   *start;

    uint32_t tlen;

    memset(buf, 0, 1024);

    tlen = sizeof(frame_header_t) + sizeof(start_measure_t);

    header = (frame_header_t *)buf;

    strcpy(header->FrameSync, FRAME_SYNC_STRING);

    header->TotalLength = tlen;

    start = (start_measure_t *) (buf + sizeof(frame_header_t));

    start->cmd = CMD_HOST_START_MEASURE;

    start->len = sizeof(start_measure_t) - 12;

    start->Ctrl.OtdrMode = 1;

    start->Ctrl.OtdrOptMode = 0;

    start->Ctrl.RSVD = 0;

    start->Ctrl.EnableRefresh = 1;

    start->Ctrl.RefreshPeriod_ms = 1000;

```



---

```

    start->State.Lambda_nm = 1550;

    start->State.MeasureLength_m = measurelen;

    start->State.PulseWidth_ns = pw;

    start->State.MeasureTime_ms = 15000;

    start->State.n = 1.4685;

    start->State.EndThreshold = 5.0;

    start->State.NonReflectThreshold = 0;

    printf("Start Measure %dm-%dns\n", measurelen, pw);

    return send(s, buf, tlen, 0);
}

// process data
int get_final = 0;

int ProcessData(char pbuf[], uint32_t len, char str[])
{
    otdr_state_t    *state;

    uint32_t    cmd;

    time_t rawtime;

    struct tm * timeinfo;

    char timE [80];

    char filename[80];

    time (&rawtime);

    timeinfo = localtime(&rawtime);

    strftime(timE, 80, "%Y-%m-%d %I:%M:%S", timeinfo);

    memset(filename, 0, sizeof filename);

    strcpy(filename, timE);

    strcat(filename, ".txt");

    cmd = *(uint32_t*)pbuf;

    switch(cmd)

    {
        case CMD_RESPONSE_STATE:

            {
                uint32_t code;

                state = (otdr_state_t*)pbuf;

```

---

```

        code = state->StateCode;

        printf("\n%s : OTDR State code : %d\n", timeE, code);

        memset(pbuf, 0, 100);

        return code;
    }

case CMD_DSP_UPLOAD_ALL_DATA:
    {
        FILE *fp;

        printf("%s : Received all final data : %d\n", timeE, len);

        fp = fopen("OtdrAllData.hex", "w");

        if (NULL == fp)
        {
            printf("Unable to open file\n");
        }
        else
        {
            fwrite(pbuf, 1, len, fp);

            fclose(fp);

            printf("Data      successfully      saved      to      file
'OtdrAllData.hex'\n");
        }

        memset(pbuf, 0, 100);

        get_final = 1;

        return len;
    }

case CMD_DSP_UPLOAD_REF_DATA:
    {
        otdr_ref_t *ref;

        ref = (otdr_ref_t*)pbuf;

        printf("%s : receive refresh data, datanum = %d\n", timeE,
ref->OtdrData.DataNum);

```

---

```

        memset(pbuf, 0, 100);

        return len;
    }

    default:
    {
        printf("cmd : 0x%x  len = %d\n", cmd, len);

        return 1;
    }
}

// measure length and pulse width
const int ml[3] = {60000, 100000, 180000};
const int pw[2][3] =
{
    {80, 640, 1280},
    {640, 5120, 5120},
};

// main function
int main(int argc, char *argv[])
{
    char str[64];

    int pulsewidth;

    int i, count, sock, rxlen, curlen, totalrxlen, size;
    struct sockaddr_in dst;
    frame_header_t *header;

    if (argc != 2)
    {
        printf("Usage: %s otdr_ip\n", argv[0]);

        return 1;
    }
}

```

---

```

size = sizeof(dst);

bzero(&dst, size);

dst.sin_family = AF_INET;

dst.sin_port = htons(5000);

dst.sin_addr.s_addr = inet_addr(argv[1]);

sock = socket(PF_INET, SOCK_STREAM, 0);

if (sock < 0)
{
    perror("socket");

    return -1;
}

printf("socket ok\n");

// set receive buffer size to 256K

count = 256*1024;

setsockopt(sock, SOL_SOCKET, SO_RCVBUF, &count, sizeof(count));

if (connect(sock, (struct sockaddr*)&dst, size) < 0)
{
    perror("connect");

    printf("errno = %d\n", errno);

    return -1;
}

printf("connect ok\n");

// start measurement and receive data

for(i = 0; i < 3; i++)
{
    for(pulsewidth = pw[0][i]; pulsewidth <= pw[1][i]; pulsewidth *= 2)
    {
        HostStartMeasure(sock, ml[i], pulsewidth);

        get_final = 0;
    }
}

```

RETRY:

---

```

        memset(str, 0, sizeof str);

        snprintf(str, 64, "d%dm_%dns.txt", ml[i], pulsewidth);

        curlen = recv(sock, buf, sizeof(frame_header_t), 0);    // Receive
frame header, obtain length information
        if (curlen <= 0)
        {
            printf("%d : recv error --> %s\n", __LINE__, strerror(errno));
            return -1;
        }

        // check for frame start string
        if(strcmp(buf, FRAME_SYNC_STRING) != 0)
        {
            printf("frame sync string not found\n");
            return -1;
        }

        // check for length
        header = (frame_header_t*)buf;
        totalrxlen = header->TotalLength;
        totalrxlen -= sizeof(frame_header_t);
        if (totalrxlen < 12)
        {
            printf("total rxlen error\n");
            return -1;
        }

        // rcv remainder
        rxlen = 0;
        while(rxlen < totalrxlen)
        {
            curlen = recv(sock, buf+rxlen, totalrxlen-rxlen, 0);
            if(curlen <= 0)

```

---

```

        {
            printf("%d : recv error --> %s\n", __LINE__, strerror(errno));
            return -1;
        }

        rxlen += curlen;
    }

    rxlen = ProcessData(buf, rxlen, str);
    NetworkIdle(sock);

    if(get_final == 0)        goto RETRY;

    sleep(1);
}

}

printf("Measurement complete\n");
close(sock);
return 0;
}

```

Compile this file on a Linux system, connect the host to the OTDR module, and run the programme to obtain the following result:

```

pheigenbaum@ubuntu:~/my-projects/gl3800m$ ./a.out 192.168.1.249
socket ok
connect ok
Start Measure 60000m-80ns
22 November 2012 10:33:30 : OTDR State code : 0
22 November 2012 10:33:32 : received refresh data, datanum = 29392
22 November 2012 10:33:33 : received refresh data, datanum = 29392
22 November 2012 10:33:34 : received refresh data, datanum = 29392
22 November 2012 10:33:36 : received refresh data, datanum = 29392
22 November 2012 10:33:37 : received refresh data, datanum = 29392
22 November 2012 10:33:38 : received refresh data, datanum = 29392
22 November 2012 10:33:39 : received refresh data, datanum = 29392

```

---

22 November 2012 10:33:40 : received refresh data, datanum = 29392  
22 November 2012 10:33:41 : received refresh data, datanum = 29392  
22 November 2012 10:33:43 : received refresh data, datanum = 29392  
22 November 2012 10:33:44 : received refresh data, datanum = 29392  
22 November 2012 10:33:45 : received refresh data, datanum = 29392  
22 November 2012 10:33:46 : received refresh data, datanum = 29392  
22 November 2012 10:33:47 : received refresh data, datanum = 29392  
22 November 2012 10:33:49 : Received final complete data: 58904  
Data successfully saved to the file 'OtdrAllData.hex'